

Guide d'exploitation

Guide d'exploitation

1. Objet du document

Ce document décrit les opérations courantes nécessaires à l'exploitation de localeo-backend.

Il couvre :

- la configuration par variables d'environnement ;
- la gestion des secrets ;
- la gestion des API keys ;
- les endpoints batch ;
- les vérifications de santé ;
- les logs ;
- les opérations courantes d'exploitation ;
- les points d'attention en mise en production.

Ce guide est volontairement orienté runbook.

2. Emplacement des éléments d'exploitation

Documentation

- guide principal : docs/ops/exploitation.md
- aide opérationnelle back-office : docs/ops/back-office-aide-opérationnelle.md
- procédures opérationnelles back-office : docs/ops/procédures-opérationnelles.md
- ancien guide détaille : docs/ops/back-office-localeo-guide-formation.md
- configuration d'un environnement : docs/ops/configuration-environnement.md
- initialisation API keys : docs/ops/v111_api_keys_initialisation.md

Configuration applicative

- source de verite runtime : app/config.py
- guide détaille : docs/ops/configuration-environnement.md
- configuration locale : .env puis .env.local selon LOCALEO_ENV
- configuration test/recette : injecter les variables dans l'environnement cible ; .env.test n'est pas chargé automatiquement par le backend

Entrée applicative

- application FastAPI : app/main.py
- healthcheck : GET /sante

Scripts SQL

- répertoire : sql/

3. Variables d'environnement à gérer

3.1 Base de données et sécurité centrale

LOCALEO_DATABASE_URL

But : connexion PostgreSQL.

Exemple :

LOCALEO_DATABASE_URL=postgresql+psycopg://localeo:password@db-host:5432/localeo

Compatibilité Render :

- l'application accepte aussi DATABASE_URL, nom souvent fourni automatiquement par Render ;
- si les deux variables existent, LOCALEO_DATABASE_URL est prioritaire ;
- en production, ne jamais utiliser la valeur locale localhost:5432.

LOCALEO_FRONT_FEEDBACK_PRESTATION_URL_TEMPLATE

But : URL front appelée depuis l'email prestation_utilisee pour collecter le feedback client post-prestation.

Exemple :

LOCALEO_FRONT_FEEDBACK_PRESTATION_URL_TEMPLATE=https://localeo.app/feedback-prestation?token={token}

Contraintes :

- doit contenir les placeholders {token} et {note} ;
- le token expire après 30 jours par défaut.

LOCALEO_QR_COFFRET_KEY_ID

But : identifiant de la clé active utilisée pour signer les QR achat_coffret.

Exemple :

LOCALEO_QR_COFFRET_KEY_ID=coffret-v2

LOCALEO_QR_COFFRET_KEYRING

But : trousseau des clés valides pour les QR achat_coffret, au format JSON key_id -> secret.

Exemple :

LOCALEO_QR_COFFRET_KEYRING={"coffret-v1":"ancien-secret","coffret-v2":"nouveau-secret"}

Compatibilité :

- en configuration simple, LOCALEO_QR_COFFRET_SECRET reste accepté comme fallback ;

- en production, preferer LOCALEO_QR_COFFRET_KEYRING ;
- le QR commerçant est decommissionné et ne dispose plus de keyring dédié.

Exigences ops :

- ne jamais laisser les valeurs par défaut en environnement exposé ;
- utiliser des secrets longs, aléatoires, stockés dans le coffre de secrets ;
- s'assurer que LOCALEO_QR_COFFRET_KEY_ID existe bien dans LOCALEO_QR_COFFRET_KEYRING ;
- ne retirer une ancienne clé du keyring qu'après extinction effective des QR coffrets correspondants.

Authentification commerçant

Variables à configurer :

- LOCALEO_BCRYPT_ROUNDS : coût bcrypt, valeur cible 12 ;
- LOCALEO_PASSWORD_MIN_LENGTH : longueur minimale, valeur cible 12 ;
- LOCALEO_PASSWORD_RESET_TOKEN_TTL_MINUTES : durée du lien de réinitialisation, valeur cible 60 ;
- LOCALEO_PASSWORD_INIT_TOKEN_TTL_HOURS : durée du lien d'initialisation, valeur cible 24 ;
- LOCALEO_AUTH_RATE_LIMIT_WINDOW_SECONDS : fenêtre de rate limiting, valeur cible 900 ;
- LOCALEO_AUTH_LOGIN_MAX_FAILURES_PER_WINDOW : échecs login par fenêtre, valeur cible 5 ;
- LOCALEO_AUTH_PASSWORD_RESET_MAX_REQUESTS_PER_WINDOW : demandes reset par fenêtre, valeur cible 5 ;
- LOCALEO_AUTH_ACCOUNT_LOCK_MAX_FAILURES : seuil de verrouillage compte, valeur cible 5 ;
- LOCALEO_FRONT_COMMERCANT_PASSWORD_INIT_URL_TEMPLATE : URL front contenant {token} pour initialiser le premier mot de passe ;
- LOCALEO_FRONT_COMMERCANT_PASSWORD_RESET_URL_TEMPLATE : URL front contenant {token} pour réinitialiser le mot de passe.

Points d'exploitation :

- surveiller événements_audit pour auth.login.failed, account.locked, password.reset.requested ;
- le déverrouillage compte est manuel depuis le back-office ;
- les tables identifiants_commerçant, tokens_accès_commerçant et rate_limits_authentification sont ajoutées par la migration Epic 10 ;
- les tokens d'initialisation/réinitialisation ne sont stockés que sous forme hashée.

3.2 Paiement Stripe

LOCALEO_STRIPE_SECRET_KEY

But : clé API Stripe côté serveur.

Exigences ops :

- utiliser une clé adaptée à l'environnement ;
- ne jamais mélanger clés test et live ;

- vérifier la cohérence avec le frontend consommateur et le compte Stripe ciblé.

Droits minimaux recommandés :

- le backend actuel utilise Stripe pour créer une Checkout Session côté serveur ;
- si une restricted key Stripe est utilisée, donner au minimum le droit Checkout Sessions: Write ;
- ne pas ouvrir Customers, PaymentIntents, Refunds, Products, Prices, Subscriptions ou Payouts tant qu'aucun besoin explicite n'existe dans le code ;
- tester d'abord avec ce périmètre minimal, puis n'ajouter un droit complémentaire qu'en cas d'erreur Stripe explicite sur une ressource manquante.

Point d'attention :

- la validation de webhook n'utilise pas un droit API additionnel ; chaque destination possède son propre secret de signature.
- les deux destinations utilisent les événements Stripe API v1 au format Snapshot ; ne pas sélectionner les événements v2 Thin sans migration du contrat de payload et des handlers.

LOCALEO_STRIPE_WEBHOOK_SECRET

But : validation des événements du compte plateforme reçus sur POST /public/stripe/webhook.

Exigences ops :

- événements à activer : checkout.session.completed, checkout.session.expired, checkout.session.async_payment_failed, payment_intent.succeeded, payment_intent.payment_failed, charge.succeeded, charge.failed, charge.refunded, refund.created, refund.updated, refund.succeeded, refund.failed, transfer.created, transfer.updated et transfer.reversed ;
- secret spécifique à la destination plateforme configurée dans Stripe ;
- à mettre à jour si cette destination est recrée côté Stripe.

LOCALEO_STRIPE_CONNECT_WEBHOOK_SECRET

But : validation des événements émis pour les comptes connectés reçus sur POST /public/stripe-connect/webhook.

Exigences ops :

- activer uniquement account.updated ;
- sélectionner la source ? comptes connectés ? dans Stripe ;
- à mettre à jour si cette destination est recrée côté Stripe.

3.3 URLs frontend et liens contractuels

LOCALEO_FRONT_SUCCESS_URL

But : redirection après paiement réussi.

LOCALEO_FRONT_CANCEL_URL

But : redirection après annulation du paiement.

LOCALEO_FRONT_COMMANDE_URL_TEMPLATE

But : lien envoyé à l'acheteur professionnel pour gérer sa commande.

Format attendu :

LOCALEO_FRONT_COMMANDE_URL_TEMPLATE=https://front.example/achats/{achat_id}?management_token={m

LOCALEO_FRONT_ACTIVATION_URL_TEMPLATE

But : lien d'activation envoyé au bénéficiaire.

Format attendu :

LOCALEO_FRONT_ACTIVATION_URL_TEMPLATE=https://front.example/activation?achat_id={achat_id}&coffret_in

Exigences ops :

- ne pas laisser pointer vers un mauvais environnement ;
- vérifier la cohérence entre backend et frontend avant ouverture au public.

3.4 Service image

LOCALEO_IMAGE_API_BASE_URL

But : construire les URLs absolues de vignette à partir de image_uri.

Exemple :

LOCALEO_IMAGE_API_BASE_URL=https://api.example.com

Remarque :

- les URLs construites pointent vers /api/images/... si le contrat frontend l'exige.

LOCALEO_EMAIL_ASSET_BASE_URL

But : construire les URLs absolues des images incluses dans les emails, notamment le logo Localeo.

Exemple :

LOCALEO_EMAIL_ASSET_BASE_URL=https://api.example.com

Points de controle :

- l'URL https://api.example.com/email-assets/localeo.png doit répondre publiquement en 200 image/png ;
- ne pas utiliser une URL localhost pour un email envoyé à un vrai destinataire ;
- ce paramètre est séparé de LOCALEO_IMAGE_API_BASE_URL, qui peut pointer vers le frontend ou vers un proxy image.

LOCALEO_IMAGE_MAX_SIZE_BYTES

But : taille maximale autorisée pour un upload image.

Exemple :

LOCALEO_IMAGE_MAX_SIZE_BYTES=153600

Recommandation :

- garder une limite basse pour contenir la taille de la table media_assets.

3.4B Gestion documentaire

LOCALEO_DOCUMENT_STORAGE_PROVIDER

But : choisir le provider de stockage documentaire.

Valeurs supportees :

- LOCAL pour le developpement ;
- SCALEWAY_OBJECT_STORAGE ou S3 pour la production via API S3 compatible.

LOCALEO_DOCUMENT_STORAGE_LOCAL_DIR

But : repertoire local utilise en developpement.

Default : storage/documents.

LOCALEO_DOCUMENT_STORAGE_S3_ENDPOINT_URL

But : endpoint S3 compatible Scaleway.

Exemple :

LOCALEO_DOCUMENT_STORAGE_S3_ENDPOINT_URL=https://s3.fr-par.scw.cloud

LOCALEO_DOCUMENT_STORAGE_S3_REGION

But : region documentaire.

Default : fr-par.

LOCALEO_DOCUMENT_STORAGE_S3_BUCKET

But : bucket documentaire.

Default cible : localeo-archives.

LOCALEO_DOCUMENT_STORAGE_S3_PREFIX

But : prefixe applicatif dans le bucket.

Default cible : /archives.

LOCALEO_DOCUMENT_STORAGE_S3_ACCESS_KEY et LOCALEO_DOCUMENT_STORAGE_S3_SECRET_KEY

But : secrets S3 utilisés pour lire/écrire les documents.

Règle : stocker ces secrets dans l'environnement Render ou le secret manager, jamais dans Git.

LOCALEO_DOCUMENT_UPLOAD_MAX_SIZE_BYTES

But : taille maximale d'upload documentaire.

Default : 2097152 octets, soit 2 Mo.

LOCALEO_DOCUMENT_ALLOWED_MIME_TYPES

But : liste CSV des MIME types documentaires autorisés.

Default : application/pdf,text/html.

3.5 Parcours métier achats / validation

LOCALEO_ACTIVATION_EMAIL_RESEND_COOLDOWN_SECONDS

But : limiter le renvoi trop fréquent d'un email d'activation de coffret instance.

LOCALEO_MANAGEMENT_TOKEN_TTL_HOURS

But : durée de vie des management_token des achats professionnels.

LOCALEO_TRANSACTION_DURATION_SECONDS

But : durée de vie d'une transaction de validation terrain.

Impact ops :

- cette valeur agit directement sur la fenêtre pendant laquelle un commerçant peut valider une prestation.

3.6 Administration SQLAdmin

LOCALEO_ADMIN_USERNAME

LOCALEO_ADMIN_PASSWORD

LOCALEO_ADMIN_SESSION_SECRET

But : sécuriser l'accès SQLAdmin.

Exigences ops :

- ne jamais garder les valeurs par défaut ;
- générer un ADMIN_SESSION_SECRET fort ;
- stocker le mot de passe dans un secret manager.

3.7 Brevo Email / SMS

BREVO_API_KEY

Clé générique Brevo, utilisée comme fallback.

BREVO_EMAIL_API_KEY

Clé API dédiée email si séparation souhaitée.

BREVO_SMS_API_KEY

Clé API dédiée SMS si séparation souhaitée.

BREVO_SENDER_EMAIL

Expéditeur email transactionnel.

BREVO_SENDER_NAME

Nom d'expéditeur.

EMAIL_DEV_MODE

Mode de développement email.

BREVO_SMS_SENDER

Nom d'émetteur SMS.

SMS_DEV_MODE

Mode de développement SMS.

Exigences ops :

- en production, désactiver les dev modes si l'envoi réel est souhaité ;
- valider les identités expéditeur côté Brevo ;
- surveiller la cohérence entre clés email/SMS et environnement visé.

3.8 Live tracking WebPush / VAPID

Deux canaux WebPush sont configurés séparément :

- COMMERCE pour le live tracking envoyé à l'application commerçant.
- LIVE pour les alertes internes de la PWA Localeo Live / exploitation.

Les anciennes clés génériques LOCALEO_WEBPUSH_* et

LOCALEO_FEATURE_LIVE_TRACKING_WEBPUSH_ENABLED restent lues en fallback, mais ne sont plus la cible pour les nouveaux environnements.

Canal commerçant

Feature flag :

LOCALEO_WEBPUSH_COMMERCE_ENABLED=false

Effet :

- si false, aucune outbox WebPush commercant n'est creee ;
- si false, le batch WebPush commercant retourne feature_disabled et n'envoie rien.

VAPID :

LOCALEO_WEBPUSH_COMMERCANT_VAPID_PRIVATE_KEY=...

LOCALEO_WEBPUSH_COMMERCANT_VAPID_PUBLIC_KEY=...

LOCALEO_WEBPUSH_COMMERCANT_VAPID_SUBJECT=mailto:support@localeo.fr

Deep link opaque :

LOCALEO_WEBPUSH_COMMERCANT_DEEPLINK_TEMPLATE=localeo://commercant/live-tracking/{opaque_reference}

LOCALEO_WEBPUSH_COMMERCANT_DEEPLINK_TTL_HOURS=168

Contraintes :

- le template doit contenir {opaque_reference} ;
- le payload client ne doit pas exposer achat_id ou coffret_id.

Retention :

LOCALEO_WEBPUSH_COMMERCANT_ABONNEMENT_REVOQUE_RETENTION_DAYS=90

LOCALEO_WEBPUSH_COMMERCANT_SORTANT_RETENTION_DAYS=90

Canal Localeo Live

Feature flag :

LOCALEO_WEBPUSH_LIVE_ENABLED=true

Effet :

- si false, les abonnements Live ne peuvent plus etre crees ;
- si false, aucune outbox WebPush Live n'est creee ;
- si false, le batch WebPush exploitation retourne feature_disabled.

VAPID :

LOCALEO_WEBPUSH_LIVE_VAPID_PRIVATE_KEY=...

LOCALEO_WEBPUSH_LIVE_VAPID_PUBLIC_KEY=...

LOCALEO_WEBPUSH_LIVE_VAPID_SUBJECT=mailto:support@localeo.fr

Retention :

LOCALEO_WEBPUSH_LIVE_ABONNEMENT_REVOQUE_RETENTION_DAYS=90

LOCALEO_WEBPUSH_LIVE_SORTANT_RETENTION_DAYS=90

La purge opérationnelle exposee aujourd'hui par /protected/maintenance/webpush/purger concerne le canal commercant. Les cle de retention Live sont reservees pour une purge Live dediee.

Exigences ops communes :

- stocker les cle privees exclusivement dans le secret manager ;
- ne jamais committer ni afficher les cle privees dans les logs ;

- separer les cles par environnement et, si possible, par canal ;
- si la cle privee est absente, le provider WebPush fonctionne en mode no-op de developpement (dev_skipped).

Points d'exploitation :

- surveiller les vues SQLAlchemy WebPush sortants, Abonnements WebPush commercants, WebPush exploitation sortants et Abonnements WebPush exploitation ;
- lancer le batch WebPush comme les batchs email/SMS ;
- lancer le batch WebPush exploitation automatiquement comme les batchs email/SMS ;
- purger regulierement les notifications WebPush et les traces d'abonnements revoques du canal commercant ;
- un retour provider 404 ou 410 desactive l'abonnement et purge ses secrets d'envoi.

3.9 Logging

LOCALEO_LOG_LEVEL

Niveau global du logger racine.

LOCALEO_LOG_FORMAT

Formater text ou json. Le JSON est requis sur test partage, preproduction et production ; le texte reste le default local.

LOCALEO_LOG_SCHEMA_VERSION

Version du contrat structure. Valeur courante : 1.

LOCALEO_LOG_SERVICE_NAME

Nom de service ajoute a chaque evenement. Default : localeo-backend.

LOCALEO_LOG_USER_AGENT_MAX_LENGTH

Taille maximale du User-Agent assaini. Default : 512.

LOCALEO_SQL_LOG_LEVEL

Niveau des logs SQLAlchemy.

LOCALEO_USE_CASES_LOG_LEVEL

Niveau des logs des use cases sous le namespace canonique localeo.use_cases.*.

LOCALEO_DOMAINE_LOG_LEVEL

Pilote les evenements explicites localeo.domaine.decisions et localeo.domaine.business_rules. Les methodes du domaine ne sont pas instrumentees automatiquement, afin de limiter le bruit et les risques de divulgation de données metier.

Les evenements actuellement couverts concernent notamment les transitions de reversement et de notifications, la consommabilite des coffrets, l'eligibilite Stripe Connect, les scopes de cles API et le statut public des virements. Les decisions et regles acceptees sont en DEBUG; les regles refusees sont en WARNING.

LOCALEO_INFRASTRUCTURE_LOG_LEVEL

Niveau des logs localeo.infrastructure.* pour les repositories, Unit of Work, gateways et providers externes.

LOCALEO_INVARIANTS_LOG_LEVEL

Niveau des logs localeo.domaine.invariants.

Les invariants valides sont en DEBUG. Une valeur invalide ou une transition interdite est emise en WARNING avant l'exception metier. Les valeurs brutes d'un email, telephone, code postal ou token ne sont pas journalisees ; seules des caracteristiques non sensibles comme la longueur peuvent l'etre.

LOCALEO_LOG_SLOW_USE_CASE_MS

Seuil en millisecondes au-dela duquel un Use Case genere l'evenement use_case.slow au niveau WARNING. Valeur par default : 2000.

LOCALEO_SLOW_CALL_MS reste un alias de compatibilite si la nouvelle variable n'est pas definie.

LOCALEO_AUDIT_EXTERNAL_JSONL_PATH

Chemin optionnel d'un journal JSONL append-only pour dupliquer les evenements financiers critiques hors base principale.

Exemple :

LOCALEO_AUDIT_EXTERNAL_JSONL_PATH=/var/log/localeo/reversements-audit.jsonl

Le chemin doit pointer vers un support dont la sauvegarde/restauration est distincte de la base applicative. Si la variable est absente, seul l'audit en base est ecrit.

Recommandation ops :

- en diagnostic : SQL en DEBUG, use cases en INFO, invariants en DEBUG ;
- en production nominale : reduire les logs SQL si le volume devient trop important.

4. Secrets à gérer

Secrets sensibles du projet :

- LOCALEO_QR_COFFRET_KEYRING

- LOCALEO_STRIPE_SECRET_KEY
- LOCALEO_STRIPE_WEBHOOK_SECRET
- LOCALEO_STRIPE_CONNECT_WEBHOOK_SECRET
- BREVO_API_KEY
- BREVO_EMAIL_API_KEY
- BREVO_SMS_API_KEY
- LOCALEO_WEBPUSH_COMMERCANT_VAPID_PRIVATE_KEY
- LOCALEO_WEBPUSH_LIVE_VAPID_PRIVATE_KEY
- LOCALEO_ADMIN_PASSWORD
- LOCALEO_ADMIN_SESSION_SECRET
- les API keys générées dans la table api_keys

Bonnes pratiques :

- utiliser un secret manager ou les variables d'environnement de la plateforme de déploiement ;
- ne jamais committer de valeur réelle ;
- séparer les secrets par environnement ;
- documenter une procédure de rotation pour Stripe, Brevo, VAPID, admin et API keys.

Rotation des cles QR

Procedure recommandee :

- Ajouter une nouvelle entree dans le keyring coffret sans retirer l ancienne.
- Basculer LOCALEO_QR_COFFRET_KEY_ID vers la nouvelle cle active.
- Redeployer l application.
- Laisser vivre les anciens QR coffrets tant que leur kid reste present dans le keyring.
- Regenerer si necessaire les QR coffrets longue duree.
- Retirer l ancienne entree du keyring uniquement quand plus aucun QR legitime ne doit dependre de cette cle.

Exemple coffret :

LOCALEO_QR_COFFRET_KEY_ID=coffret-v2

LOCALEO_QR_COFFRET_KEYRING={"coffret-v1":"ancien-secret","coffret-v2":"nouveau-secret"}

5. API keys d'exploitation

5.1 Principe

Les endpoints batch sont protégés par header X-API-KEY et par contrôle de scope.

Implémentation :

- app/security/api_keys.py

Scope actuellement observé dans le code :

- internal:batch

Le contrôle de scope est strict.

5.2 Endpoints protégés par API key

Emails

- POST /protected/emails/batch/envoyer
- POST /protected/emails/batch/synchroniser-statuts
- POST /protected/emails/{email_id}/relancer

SMS

- POST /protected/sms/batch/envoyer
- POST /protected/sms/batch/synchroniser-statuts
- POST /protected/sms/{sms_id}/relancer

WebPush

- POST /protected/webpush/batch/envoyer
- POST /protected/webpush-exploitation/batch/envoyer
- POST /protected/maintenance/webpush/purger

5.3 Initialisation des API keys

Script disponible :

- generer_api_key.ps1
(/d:/OneDrive/Bureau/Documents/GitHub/localeo-backend/docs/ops/generer_api_key.ps1)

Script PowerShell de génération :

```
$raw = "loc_batch_" + [guid]::NewGuid().ToString("N")
$prefix = $raw.Substring(0, 8)
$sha = [System.Security.Cryptography.SHA256]::Create()
$bytes = [System.Text.Encoding]::UTF8.GetBytes($raw)
$hashBytes = $sha.ComputeHash($bytes)
$hash = -join ($hashBytes | ForEach-Object { $_.ToString("x2") })
"RAW=$raw"
"PREFIX=$prefix"
"HASH=$hash"
```

Utilisation :

- conserver RAW dans le coffre de secrets ; c'est la valeur à envoyer dans le header X-API-KEY ;
- insérer PREFIX dans api_keys.key_prefix ;

- insérer HASH dans api_keys.key_hash ;
- choisir le scope minimal nécessaire, par exemple internal:batch.

Exemple SQL :

```
INSERT INTO api_keys (  
id,  
label,  
key_prefix,  
key_hash,  
scope,  
active  
) VALUES (  
gen_random_uuid(),  
'batch emails',  
'<PREFIX>',  
'<HASH>',  
'internal:batch',  
true  
);
```

Après insertion :

- la clé devient visible dans SQLAdmin sur API keys ;
- les champs label, scope et active sont éditables depuis l'interface d'administration ;
- key_prefix et key_hash restent non éditables et la clé brute n'est jamais réaffichée.

5.4 Rotation d'une API key

Procédure recommandée :

- Générer une nouvelle clé brute.
- Calculer son hash SHA-256 et son préfixe.
- Insérer la nouvelle ligne dans api_keys avec le bon scope et active=true.
- Déployer la nouvelle clé côté ordonnanceur / plateforme d'appel.
- Vérifier les batchs avec la nouvelle clé.
- Désactiver l'ancienne clé (active=false).

Recommandation :

- éviter de supprimer immédiatement l'ancienne clé ;
- faire une courte période de recouvrement pour éviter une rupture batch.

6. Endpoints utiles en exploitation

6.1 Santé applicative

GET /sante

But : vérifier que l'application répond HTTP.

Exemple :

```
Invoke-RestMethod -Method Get -Uri "http://localhost:8000/sante"
```

Réponse attendue :

```
{"statut":"OK"}
```

Remarque :

- cet endpoint ne prouve pas à lui seul la bonne santé de PostgreSQL, Stripe ou Brevo ;
- il confirme principalement la disponibilité du process HTTP.

6.2 Batch emails

Envoi

```
Invoke-RestMethod -Method Post `
```

```
-Uri "http://localhost:8000/protected/emails/batch/envoyer?limit=100" `
```

```
-Headers @{ "X-API-KEY" = "<RAW_KEY>" }
```

Synchronisation des statuts

```
Invoke-RestMethod -Method Post `
```

```
-Uri "http://localhost:8000/protected/emails/batch/synchroniser-statuts?limit=100" `
```

```
-Headers @{ "X-API-KEY" = "<RAW_KEY>" }
```

6.3 Batch SMS

Envoi

```
Invoke-RestMethod -Method Post `
```

```
-Uri "http://localhost:8000/protected/sms/batch/envoyer?limit=100" `
```

```
-Headers @{ "X-API-KEY" = "<RAW_KEY>" }
```

Synchronisation des statuts

```
Invoke-RestMethod -Method Post `
```

```
-Uri "http://localhost:8000/protected/sms/batch/synchroniser-statuts?limit=100" `
```

```
-Headers @{ "X-API-KEY" = "<RAW_KEY>" }
```

6.4 Batch WebPush

Envoi

```
Invoke-RestMethod -Method Post `
```

```
-Uri "http://localhost:8000/protected/webpush/batch/envoyer?limit=100" `
-Headers @{ "X-API-KEY" = "<RAW_KEY>" }
Invoke-RestMethod -Method Post `
-Uri "http://localhost:8000/protected/webpush-exploitation/batch/envoyer?limit=100" `
-Headers @{ "X-API-KEY" = "<RAW_KEY>" }
```

Purge retention

```
Invoke-RestMethod -Method Post `
-Uri "http://localhost:8000/protected/maintenance/webpush/purger" `
-Headers @{ "X-API-KEY" = "<RAW_KEY>" }
```

6.5 Batch purge sessions commercant

La retention par default est configuree via LOCALEO_COMMERCANT_SESSION_PURGE_RETENTION_DAYS.
Le parametre retention_days permet de surcharger ponctuellement cette valeur.

```
Invoke-RestMethod -Method Post `
-Uri
"http://localhost:8000/protected/maintenance/sessions-commercant/purger?retention_days=30"
`
-Headers @{ "X-API-KEY" = "<RAW_KEY>" }
```

Le batch supprime les sessions commercant expirees ou revoquees avant la date limite calculee.
Les evenements d'audit qui pointent vers une session purgee sont conserves, avec session_id remis a NULL.

6.6 Script postdatage activites locales

Le script scripts/postdate_activites_locales.py permet de decaler les dates des lignes activites_locales d'un nombre de jours donne.

Il utilise la configuration applicative de connexion base :

- LOCALEO_DATABASE_URL en priorite ;
- DATABASE_URL en fallback ;
- base locale par default uniquement en environnement local.

Dates decalees par default :

- date_evenement ;
- date_publication ;
- date_expiration_publication.

date_creation n'est pas modifiee par default. Elle n'est decalee que si l'option --include-date-creation est fournie.

Regle de securite :

- le script refuse l'operation si le decalage place date_evenement ou date_publication au-dela de la date du moment ;
- si --include-date-creation est utilise, date_creation est aussi controlee ;
- date_expiration_publication n'est pas bloquee par cette regle, car une activite publiee peut legitiement avoir une expiration future.

Dry-run obligatoire par default :

```
python scripts\postdate_activites_locales.py --days 15
```

Application reelle :

```
python scripts\postdate_activites_locales.py --days 15 --apply
```

Filtres utiles :

```
python scripts\postdate_activites_locales.py --days 15 --niveau-visibilite PUBLIC
```

```
python scripts\postdate_activites_locales.py --days 15 --type-activite  
ACHAT_COFFRET_CONFIRME
```

```
python scripts\postdate_activites_locales.py --days 15 --ville-id <uuid>
```

```
python scripts\postdate_activites_locales.py --days 15 --commercant-id <uuid>
```

```
python scripts\postdate_activites_locales.py --days 15 --coffret-id <uuid>
```

Reculer les dates est refuse par default. Pour un besoin de correction explicite :

```
python scripts\postdate_activites_locales.py --days -3 --allow-negative --apply
```

Avant toute execution en production :

- lancer d'abord sans --apply ;
- verifier le nombre d'activites cibles et les bornes de dates affichees ;
- limiter le perimetre avec un filtre si l'operation ne concerne pas tout le feed ;
- conserver la sortie console dans le ticket d'exploitation.

7. Procédures d'exploitation courantes

7.1 Démarrage local / environnement d'intégration

Pré-requis :

- PostgreSQL disponible ;
- variables d'environnement renseignées ;
- schéma SQL à jour.

Étapes :

- Charger les variables d'environnement.
- Appliquer les scripts SQL nécessaires.
- Démarrer l'application FastAPI.

- Vérifier GET /sante.
- Vérifier l'accès SQLAdmin.

7.2 Mise à jour de configuration

Procédure recommandée :

- Préparer les nouvelles variables d'environnement.
- Vérifier la cohérence fonctionnelle : Stripe, Brevo, URLs frontend, admin.
- Déployer les nouvelles variables.
- Redémarrer l'application si nécessaire.
- Vérifier la santé HTTP et les flux critiques.

7.3 Rotation des secrets Stripe / Brevo

Procédure type :

- Créer le nouveau secret côté provider.
- Déployer la nouvelle variable d'environnement.
- Redémarrer l'application si nécessaire.
- Vérifier un flux réel ou de test.
- Révoquer l'ancien secret côté provider.

Point d'attention :

- pour LOCALEO_STRIPE_WEBHOOK_SECRET, toute recreation du webhook doit être reportée dans la config backend.

7.4 Mise à jour du schéma SQL

Le projet contient des scripts versionnés dans sql/.

Scripts structurants récents :

- v96_api_keys.sql
- v108_email_status_sync.sql
- v109_sms_status_tracking.sql
- v110_coffret_instance_schema_catchup.sql
- v111_prix_montant_euros.sql
- v112_management_token_achats.sql
- v113_media_assets.sql
- v114_seed_media_assets_boutiques.sql
- v153_gestion_documentaire.sql

Procédure recommandée :

- identifier les scripts non appliqués ;
- sauvegarder la base si environnement sensible ;
- appliquer les scripts dans l'ordre ;

- redémarrer l'application si le modèle applicatif évolue ;
- contrôler SQLAdmin et les endpoints critiques.

Point d'attention :

- Base.metadata.create_all(...) existe encore au bootstrap, mais ne remplace pas une vraie stratégie de migration.

7.5 Vérification des emails sortants

Procédure :

- vérifier qu'un use case a bien créé des lignes dans emails_sortants ;
- lancer /protected/emails/batch/envoyer ;
- contrôler les statuts ;
- lancer /protected/emails/batch/synchroniser-statuts si nécessaire ;
- relancer un email précis via /protected/emails/{email_id}/relancer si besoin.

7.6 Vérification des SMS sortants

Procédure :

- vérifier qu'un use case a bien créé des lignes dans sms_sortants ;
- lancer /protected/sms/batch/envoyer ;
- contrôler les statuts ;
- lancer /protected/sms/batch/synchroniser-statuts si nécessaire ;
- relancer un SMS précis via /protected/sms/{sms_id}/relancer si besoin.

Remarque :

- certains SMS restent volontairement bloqués tant que l'email associé n'est pas confirmé comme envoyé.

7.7 Verification des WebPush sortants

Procedure :

- verifier que LOCALEO_WEBPUSH_COMMERCANT_ENABLED=true si l'envoi reel est attendu ;
- verifier que LOCALEO_WEBPUSH_COMMERCANT_VAPID_PRIVATE_KEY et LOCALEO_WEBPUSH_COMMERCANT_VAPID_SUBJECT sont renseignes en production ;
- verifier qu'un achat confirme a bien cree des lignes dans webpush_sortants ;
- verifier qu'au moins un abonnement actif existe dans abonnements_webpush_commerçants pour le commerçant cible ;
- lancer /protected/webpush/batch/envoyer ;
- controler les statuts et erreurs dans la vue SQLAdmin WebPush sortants ;
- lancer /protected/maintenance/webpush/purger selon la retention configuree.

Remarque :

- sans cle VAPID, le provider reste en mode no-op de developpement et les envois sont marques dev_skipped.

7.8 Gestion SQLAdmin

Usage :

- administration des référentiels ;
- exploitation courante ;
- consultation des achats, paiements, emails, SMS, WebPush, reversements, médias et documents.

Points ops :

- changer systématiquement les identifiants par défaut ;
- surveiller l'exposition réseau de SQLAdmin ;
- réserver l'accès à un réseau d'administration si possible.

7.9 Gestion documentaire

Chemins back-office

- Gestion documentaire > Gestion documentaire : upload, liste recente, preparation HTML.
- Gestion documentaire > Gestion documentaire model view : consultation des metadonnees.
- Gestion documentaire > Rattachements documentaires : controle des rattachements metier.

Publier ou remplacer un document public

- Ouvrir /admin/gestion-documentaire.
- Uploader un PDF ou HTML.
- Choisir categorie=PUBLIC.
- Renseigner type_document et scopes.
- Choisir PUBLIE uniquement si le document doit devenir visible.
- Si un PDF public doit etre affiche en page HTML, cocher Version HTML.
- Relire la preview HTML, retoucher le contenu, puis sauvegarder et publier.

Effet attendu :

- la base stocke uniquement les metadonnees ;
- le binaire est ecrit dans le provider documentaire ;
- la mise a jour d'un document remplace le contenu stocke et conserve le meme identifiant documentaire ;
- le contenu HTML prepare depuis un PDF est rattache au PDF source.

Ajouter un document prive commercant

- Ouvrir /admin/gestion-documentaire.
- Uploader le document.
- Choisir categorie=COMMERCANT.

- Renseigner resource_type=COMMERCANT.
- Renseigner resource_id avec l'UUID du commercant.
- Laisser le document non public.

Controle :

- le commercant ne pourra voir ce document que depuis sa propre session ;
- un autre commercant ne doit ni le lister ni le telecharger ;
- la consultation cree un evenement d'audit.

Acces public et commercant

- Listing public marketplace : GET /public/documents?scope=SITE_PUBLIC.
- HTML public : GET /public/documents/{type_document}.
- PDF public : GET /public/documents/{document_id}/download.
- Documents prives commercant : GET /protected/commercants/documents.
- Telechargement prive commercant : GET /protected/commercants/documents/{document_id}/download.

Les fichiers sont servis via backend. Les URLs directes Scaleway ne doivent pas etre exposees aux utilisateurs.

8. Logs et diagnostic

8.1 Configuration actuelle

La configuration de logs est centralisée dans app/logging_config.py.

Niveaux actuels observables :

- entree et sortie des use cases : INFO ;
- parametres et resultats filtres : DEBUG ;
- regles appliquees ou ignorees : DEBUG ;
- regles rejetees et invariants invalides : WARNING ;
- erreurs techniques : ERROR ;
- Use Cases depassant LOCALEO_LOG_SLOW_USE_CASE_MS : WARNING.

Chaque trace automatique porte correlation_id, son alias request_id, use_case_invocation_id, le Use Case, l'issue et la duree. Les arguments nommes sensibles sont masques, notamment mots de passe, tokens, cles API, signatures, emails, telephones, IBAN, numeros de carte, payloads et URLs.

Chaque erreur JSON 4xx ou 5xx expose :

```
{
  "detail": "Message assaini",
```

```
"correlationId": "LOC-...",  
"request_id": "LOC-..."  
}
```

correlationId est la reference canonique a communiquer au support.

request_id est un alias deprecie. Les headers X-Correlation-ID et X-Request-ID transportent la meme valeur.

Regles d'usage :

- ne jamais ajouter un payload, une query string, un header d'autorisation ou un secret brut dans un message de log ;
 - utiliser log_business_rule(scope, rule, outcome, **details) pour une regle inline, avec applied, skipped, rejected ou failed ;
 - utiliser les logs pour le diagnostic ephemere et evenements_audit pour une preuve metier durable ;
 - conserver une seule stack trace au premier Use Case qui observe l'erreur technique, ou a la frontiere HTTP si aucun Use Case ne l'a deja journalisee ;
- les couches englobantes ne journalisent que le type et le resultat ;
- ne jamais utiliser les logs comme source de données metier ou comme mecanisme d'audit financier.

8.2 Collecte Better Stack depuis Render

- Creer dans Better Stack une source Render dans la region Allemagne.
- Dans Render, ouvrir Workspace > Integrations > Observability > Log Streams.
- Renseigner le host Syslog TLS et le token fournis par la source Better Stack.
- Verifier en test la reception d'un evenement http.request.completed et d'un couple use_case.started / use_case.succeeded partageant le meme correlation_id.
- Rechercher un correlationId retourne par une erreur API et verifier que les logs HTTP et Use Cases associes sont retrouves.
- Verifier l'absence d'email, telephone, IBAN, token, QR, cookie, header d'autorisation et valeur de query string dans les evenements.
- Configurer les retentions : test 14 jours, preproduction 30 jours, production 90 jours après validation DPO.

Alertes initiales :

- p95 superieur a 1 500 ms pendant 15 minutes avec au moins 20 executions ;
- 3 erreurs techniques en 5 minutes ou taux superieur a 5 % ;
- premier echec definitif d'un paiement, reversement, webhook Stripe ou batch critique.

Acces : exploitation et responsables techniques sur les logs assainis ; support

limite a la recherche par correlation ; finance et produit sur vues agregees.

8.3 Lecture des logs SQL

Un log ROLLBACK sur sqlalchemy.engine.Engine n'est pas forcément une erreur.

Cas nominal fréquent :

- une transaction implicite est ouverte même pour des SELECT ;
- à la fin du scope, SQLAlchemy ferme proprement la transaction via ROLLBACK ;
- cela ne signifie pas une perte de données si aucune écriture n'était en cours.

8.4 Pistes de diagnostic

Si les batchs ne partent pas

Vérifier :

- présence d'une API key valide ;
- scope internal:batch ;
- statut local des emails/SMS/WebPush ;
- dev mode actif ou non ;
- erreurs Brevo ou WebPush dans erreur_derniere.

Si les WebPush ne partent pas

Vérifier :

- LOCALEO_WEBPUSH_COMMERCANT_ENABLED=true pour le canal marchand, ou LOCALEO_WEBPUSH_LIVE_ENABLED=true pour Localeo Live ;
- LOCALEO_WEBPUSH_COMMERCANT_VAPID_PRIVATE_KEY ou LOCALEO_WEBPUSH_LIVE_VAPID_PRIVATE_KEY renseignée pour un envoi réel ;
- LOCALEO_WEBPUSH_COMMERCANT_VAPID_SUBJECT ou LOCALEO_WEBPUSH_LIVE_VAPID_SUBJECT cohérent avec l'environnement ;
- présence de lignes webpush_sortants en statut A_ENVOYER ou ECHEC_TEMPORAIRE ;
- présence d'abonnements actifs avec secrets d'envoi non purgés ;
- erreurs provider dans webpush_sortants.erreur_derniere.

Si les achats pro ne sont pas consultables

Vérifier :

- présence de management_token_hash sur l'achat ;
- expiration du token ;
- cohérence du lien frontend de commande.

Si les images ne s'affichent pas

Vérifier :

- image_uri renseigné ;
- LOCALEO_IMAGE_API_BASE_URL ;

- route /images/... accessible ;
- contenu de la table media_assets.

Si un document ne se telecharge pas

Vérifier :

- presence du document dans documents ;
- coherence de storage_provider, storage_bucket, storage_key et storage_uri ;
- variables LOCALEO_DOCUMENT_STORAGE_* ;
- acces au bucket Scaleway avec les cles configurees ;
- statut du document : un document ARCHIVE ne doit pas etre expose publiquement ou cote commercant ;
- rattachement COMMERCANT dans document_rattachements pour l'application commercant.

Si le paiement ne se valide pas

Vérifier :

- LOCALEO_STRIPE_WEBHOOK_SECRET ;
- métadonnées achat_id ;
- présence ou non d'un événement déjà traité dans paiement_events.

9. Checklist de mise en production

Avant ouverture de l'environnement :

- LOCALEO_QR_COFFRET_KEY_ID configuré ;
- LOCALEO_QR_COFFRET_KEYRING configuré, ou LOCALEO_QR_COFFRET_SECRET en fallback ;
- variables d'authentification commercant configurees (LOCALEO_BCRYPT_ROUNDS, TTL tokens, rate limiting, URLs front) ;
- identifiants SQLAdmin remplacés ;
- session secret admin remplacé ;
- clés Stripe configurées ;
- secret webhook Stripe configuré ;
- LOCALEO_FEATURE_STRIPE_CONNECT_ENABLED=false tant que le go/no-go juridique et finance EPIC 39 n'est pas valide en production ;
- LOCALEO_STRIPE_SECRET_KEY configurée sur l'environnement cible Stripe Connect ;
- LOCALEO_STRIPE_WEBHOOK_SECRET configuré pour la destination du compte plateforme ;
- LOCALEO_STRIPE_CONNECT_WEBHOOK_SECRET configuré pour la destination des comptes connectés ;
-
- LOCALEO_STRIPE_ONBOARDING_RETURN_URL=https://commercants.localeo.fr/stripe-connect/onboarding/retr
- ;

-

LOCALEO_STRIPE_ONBOARDING_REFRESH_URL=https://commercants.localeo.fr/stripe-connect/onboarding/re

- les routes onboarding Stripe Connect sont servies par l'application marchand et appellent le backend pour synchroniser ou regénérer un lien ;
- routes backend Stripe Connect disponibles :
 - GET /protected/referencement/commercants/me/stripe-connect pour consulter le statut d'onboarding marchand ;
 - POST /protected/referencement/commercants/me/stripe-connect/onboarding pour créer le compte Express si besoin et générer un lien d'onboarding ;
 - POST /protected/referencement/commercants/me/stripe-connect/synchroniser pour resynchroniser le compte connecté ;
 - POST /public/stripe/webhook pour les événements paiement, charge, remboursement et transfert du compte plateforme ;
 - POST /public/stripe-connect/webhook pour account.updated émis pour les comptes connectés ;
 - POST /protected/gestion-reversement/reversements/campagnes-stripe/lancer pour lancer une campagne de transferts Stripe ;
 - le webhook plateforme doit recevoir notamment transfer.created, transfer.updated et transfer.reversed pour confirmer les transferts ;
 - le webhook comptes connectés doit recevoir uniquement account.updated pour synchroniser l'onboarding ;
 - une campagne Stripe doit laisser les paiements reversement en TRANSFER_DEMANDE jusqu'à réception du webhook de confirmation ;
 - les demandes de remboursement créées depuis le support ou une annulation d'instance sont en mode PSP et doivent être exécutées via l'action back-office Exécuter via Stripe ;
 - aucune référence de remboursement bancaire externe ne doit être saisie manuellement ; la référence opérationnelle est le refund_id Stripe ;
 - clés Brevo configurées ;
 - stockage documentaire Scaleway configuré ;
 - bucket documentaire localeo-archives privé ;
 - taille maximale et MIME types documentaires valides ;
 - EMAIL_DEV_MODE=false si envoi réel souhaité ;
 - SMS_DEV_MODE=false si envoi réel souhaité ;
 - LOCALEO_WEBPUSH_COMMERCANT_ENABLED positionne selon l'environnement ;
 - LOCALEO_WEBPUSH_COMMERCANT_VAPID_PRIVATE_KEY configurée si le canal marchand est actif ;
 - LOCALEO_WEBPUSH_COMMERCANT_VAPID_SUBJECT configure avec un émetteur valide ;
 - LOCALEO_WEBPUSH_LIVE_ENABLED positionne selon l'environnement ;
 - LOCALEO_WEBPUSH_LIVE_VAPID_PRIVATE_KEY configurée si Localeo Live envoie réellement des WebPush ;
 - LOCALEO_WEBPUSH_LIVE_VAPID_SUBJECT configure avec un émetteur valide ;
 - URLs frontend correctes ;

- scripts SQL à jour ;
- API key batch créée ;
- healthcheck validé ;
- accès SQLAdmin restreint ;
- niveau de logs validé pour l'environnement.

10. Recommandations ops

- Ne pas utiliser les valeurs par défaut présentes dans app/config.py en environnement réel.
- Centraliser les secrets dans un secret manager.
- Versionner uniquement les exemples de configuration, jamais les secrets réels.
- Formaliser une procédure de rotation pour Stripe, Brevo, admin et API keys.
- Exécuter les batchs via un ordonnanceur externe fiable si besoin d'industrialisation.
- Réduire les logs SQL en production si le volume devient trop important.
- Documenter les scripts SQL effectivement appliqués par environnement.

11. Documents complémentaires

- Spécifications fonctionnelles
(/d:/OneDrive/Bureau/Documents/GitHub/localeo-backend/docs/specifications/fonctionnelle.md)
- Spécifications techniques
(/d:/OneDrive/Bureau/Documents/GitHub/localeo-backend/docs/specifications/technique.md)
- Architecture solution (../architecture/transverse/architecture-solution.md)
- Initialisation des API keys
(/d:/OneDrive/Bureau/Documents/GitHub/localeo-backend/docs/ops/v111_api_keys_initialisation.md)